VoteChain Private Blockchain - Developer Documentation

Project Name: VoteChain
Project Type: Private Blockchain Voting System with Fingerprint Authentication and Ephemeral Hash Security
Target Users: ~1,000 students
Timeline: 17 Sept 2025 → 13 Feb 2026
Development Team Roles: - **Core Developers (You + Rajib):** Blockchain, consensus, transaction, ephemeral hash, fingerprint integration in C++ - **Frontend Developer (Abhash):** UI/UX, React.js/Next.js, registration & voting pages - **Backend Developer (Sandesh):** Node.js + Express, API endpoints, database, blockchain integration - **Testing (Ayush):** Unit testing, load testing, security checks, multi-node testing

---

1. System Architecture

1.1 Overview

VoteChain is a **private blockchain** voting system designed to conduct secure, privacy-preserving elections. It integrates: - **Fingerprint authentication** for voter verification - **Permanent voter hashes** stored in secure backend - **Temporary ephemeral hashes** during elections to unlink votes from identities - **Blockchain ledger** for immutable vote recording - **Frontend** for voter interaction and admin dashboard - **Backend API** for integration between blockchain and frontend

1.2 Component Diagram

```
[Frontend] <---> [Backend API] <---> [Blockchain Nodes] <---> [Database: permanent hashes, voter info]
```

1.3 Key Classes (C++)

- `Transaction` : Stores vote info (tempHash, candidate selection, timestamp)
- `Block` : Index, timestamp, transactions, previousHash, hash
- `Blockchain` : Vector of Blocks, addBlock(), validateChain()
- `Node` : Peer list, broadcasting, receiving blocks
- `Fingerprint` : Capture, hash generation, ephemeral conversion

1.4 Data Flow

1. Voter registers → fingerprint captured → permanent hash created → stored in backend
2. Election starts → ephemeral temp hash generated
3. Voter votes → fingerprint scan verified → tempHash submitted to blockchain
4. Blockchain nodes validate vote and add block
5. Election ends → temp hashes discarded
6. Admin views results via frontend dashboard

2. Development Guidelines

2.1 Core Developers (C++)

Responsibilities: - Implement blockchain classes and consensus algorithm - Integrate fingerprint SDK - Generate permanent and ephemeral hashes - Ensure immutability and integrity of blockchain - Optimize code for multi-node performance and 1K users

Best Practices: - Use OOP principles: classes, encapsulation, inheritance, polymorphism - Modular code: separate registration, voting, blockchain, and node logic - Error handling for invalid fingerprints or double votes - Version control: Git with meaningful commits

2.2 Frontend Developer

Responsibilities: - Build responsive registration and voting pages - Display student info after fingerprint scan - Integrate with backend APIs - Display real-time vote count

Best Practices: - Use component-based structure in React.js - Mobile-first design - Validate inputs and handle errors gracefully - Ensure smooth UX for non-tech-savvy students

2.3 Backend Developer

Responsibilities: - Store permanent hashes and voter info securely - Provide REST API for frontend and blockchain - Handle ephemeral hash generation and validation - Manage node communication and logs

Best Practices: - Secure database with encryption - Handle API authentication and request validation - Optimize for concurrent votes (simulate 1K users) - Implement logging for debugging and auditing

2.4 Testing

Responsibilities: - Unit tests for core blockchain functions - Load testing for multi-node and 1K user simulation - Security testing: double-vote prevention, unlinkability of temp hashes - Frontend-backend integration testing

Best Practices: - Maintain test cases repository - Report bugs to respective developers with severity - Automate repetitive tests where possible

3. Fingerprint & Hash Management

3.1 Permanent Hash

- Generated at registration:

```
permanentHash = SHA256(voterID + fingerprintHash)
```

- Stored securely in backend
- Used to generate ephemeral hash only during elections

3.2 Ephemeral Hash

- Generated at start of each election:

```
tempHash = SHA256(permanentHash + electionName + electionDate +
randomSalt)
```

- Used for voting transactions on blockchain
- Discarded after election
- Prevents linking votes to permanent identity

3.3 Fingerprint Integration

- Fingerprint captured via scanner → hash template
- Temp fingerprint hash generated for election using same ephemeral logic
- Verified before allowing vote submission

---

4. Node & Consensus

- Nodes communicate via TCP/UDP sockets
- Consensus Algorithm: Proof-of-Authority (PoA) or PBFT
- Blocks propagated and validated across nodes
- Duplicate votes rejected by checking tempHash uniqueness

---

5. API Endpoints (Backend)

- `/registerVoter` : Store permanent hash and voter info
- `/verifyFingerprint` : Validate fingerprint against permanent hash
- `/generateTempHash` : Generate ephemeral hash for election
- `/castVote` : Submit vote transaction to blockchain
- `/getResults` : Query vote counts

---

6. Testing Guidelines

- Test blockchain functions: addBlock, validateChain, duplicate votes
- Test fingerprint capture and ephemeral hash logic
- Load test 1K simulated voters
- Frontend-backend integration tests
- Security checks: digital signatures, temp hash unlinkability

---

7. Documentation & Reporting

- Maintain UML diagrams, flowcharts, and class diagrams
- Document all API endpoints and usage examples

- Maintain GitHub repository with clear README, commits, and versioning
- Report bugs and fixes in issue tracker

Note: Follow coding standards, modular architecture, and document all features for ease of understanding and future maintenance.